

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 4

Disciplina: Tehnici de programare

Tema: ”Introducere în Python”

A efectuat :

st. gr.SI – 261 Grecu Gabriela

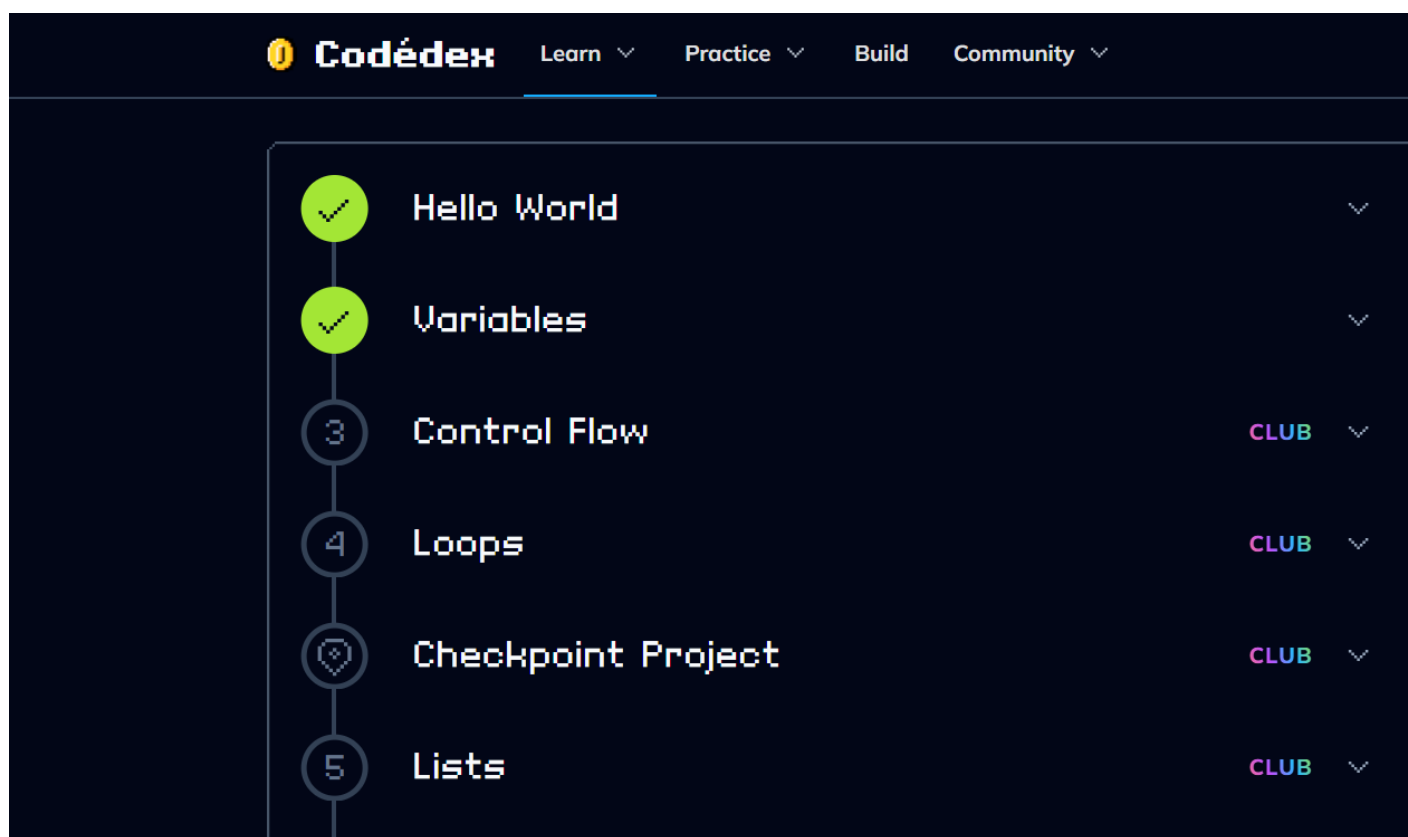
A verificat :

asistent universitar Marius Dumitrașcu

Chișinău – 2026

SARCINA LUCRĂRII

1. Prezenta lucrare de laborator se va desfășura în format interactiv, utilizând platforma educațională [Codédex \(secțiunea Python\)](#). Studenții vor realiza următoarele sarcini practice:
2. **1. Configurarea mediului de lucru și inițierea sesiunii**
3. Accesați platforma și autentificați-vă (creați un cont dacă este necesar pentru salvarea progresului).
4. Navigați către cursul de bază (*The Legend of Python*) și localizați editorul de cod (IDE) integrat în interfața web.
5. **2. Implementarea conceptelor fundamentale (Scriere de cod)** Parcurgeți modulele introductive și implementați scripturile solicitate pentru fiecare etapă. Sarcinile obligatorii includ:
6. **I/O de bază:** Utilizarea funcției `print()` pentru afișarea textului formatat la consolă.
7. **Variabile și tipuri de date:** Declararea, inițializarea și manipularea variabilelor (int, float, string, boolean).
8. **Operatori și expresii:** Evaluarea expresiilor matematice și a operatorilor logici.
9. **Structuri de control:** Implementarea logicii decizionale utilizând blocurile `if`, `elif` și `else`.



Exercise

Use the "print" function to print the line "Hello, World!"

```
script.py
1 print("Hello, World!")

iPython Shell
<script.py> output:
Hello, World!

In [1]:
```

Great job!

[Solution](#) [Submit](#)

The target of this exercise is to create a string, an integer, and a floating point number. The string should be named `mystring` and should contain the word "hello". The floating point number should be named `myfloat` and should contain the number 10.0, and the integer should be named `myint` and should contain the number 20. [View More Tutorials](#)

```
script.py
1 # change this code
2 mystring = "hello"
3 myfloat = 10.0
4 myint = 20
5
6 # testing code
7 if mystring == "hello":
8     print("String: %s" % mystring)
9 if isinstance(myfloat, float) and myfloat == 10.0:
10     print("Float: %s" % myfloat)
11 if isinstance(myint, int) and myint == 20:
12     print("Integer: %s" % myint)

iPython Shell
<script.py> output:
String: hello
Float: 10.000000
Integer: 20

In [1]:
```

Great job!

[Solution](#) [Submit](#)

Exercise

In this exercise, you will need to add numbers and strings to the correct lists using the "append" list method. You must add the numbers 1, 2, and 3 to the "numbers" list, and the words "hello" and "world" to the strings variable. [List manipulation guide](#)

You will also have to fill in the variable `second_name` with the second name in the names list, using the brackets operator `[]`. Note that the index is zero-based, so if you want to access the second item in the list, its index will be 1.

```
script.py
1 numbers = [1,2,3]
2 strings = ["hello"]
3 names = ["John", "Eric", "Jessica"]
4
5 # write your code here
6 second_name = "Eric"
7
8 # this code should write out the filled arrays and
9 # the second name in the names list (Eric).
10 print(numbers)
11 print(strings)
12 print("The second name on the names list is %s" %
    second_name)

iPython Shell
<script.py> output:
[1, 2, 3]
['hello']
The second name on the names list is 'Eric'

In [1]:
```

Great Job!

[Solution](#) [Submit](#)

Exercise

The target of this exercise is to create two lists called `x_list` and `y_list`, which contain 10 instances of the variables `x` and `y`, respectively. You are also required to create a list called `big_list`, which contains the variables `x` and `y` 10 times each, by concatenating the two lists you have created.

```
script.py
1 x = object()
2 y = object()
3
4 # TODO: change this code
5 x_list = [x] * 10
6 y_list = [y] * 10
7 big_list = x_list + y_list
8
9 print("x_list contains %d objects" % len(x_list))
10 print("y_list contains %d objects" % len(y_list))
11 print("big_list contains %d objects" % len(
    big_list))
12
13 # testing code
14 if x_list.count(x) == 10 and y_list.count(y) ==
    10:
```

Good work!

[Solution](#) [Submit](#)

Exercise

You will need to write a format string which prints out the data using the following syntax: `hello %s %s. Your current balance is $%s.44.`

```
script.py
1 data = ("John", "Doe", 53.44)
2 format_string = "Hello %s %s. Your current balance
    is $%s.44."
3
4 print(format_string % data)

iPython Shell
<script.py> output:
Hello John Doe. Your current balance is $53.44.

In [1]:
```

Great work!

[Solution](#) [Submit](#)

Exercise

Try to fix the code to print out the correct information by changing the string.

```
script.py
1 s = "Strings are awesome!"
2 # Length should be 20
3 print("Length of s - %d" % len(s))
4
5 # First occurrence of "a" should be at index 8
6 print("The first occurrence of the letter a - %d"
    % s.index("a"))
7
8 # Number of a's should be 2
9 print("a occurs %d times" % s.count("a"))
10
11 # Slicing the string into bits
12 print("The first five characters are '%s'" % s[:
    5]) # Start to 5
13 print("The next five characters are '%s'" % s[5:
    10]) # 5 to 10
```

Great work!

[Solution](#) [Submit](#)

Exercise

Change the variables in the first section, so that each if statement resolves as True.

```
script.py
1 # change this code
2 number = 16
3 second_number = 0
4 first_array = [1,2,3]
5 second_array = [1,2]
6
7 if number > 15:
8     print("1")
9
10 if first_array:
11     print("2")
12
13 if len(second_array) == 2:
14     print("3")
15
16 if len(first_array) + len(second_array) == 5:
```

Great Work!

Solution Submit

Exercise

Loop through and print out all even numbers from the numbers list in the same order they are received. Don't print any numbers that come after 237 in the sequence.

```
script.py
5 566, 597, 978, 328, 615, 953, 345,
6 399, 162, 758, 219, 918, 237, 412, 566, 826,
7 248, 866, 950, 626, 949, 687, 217,
8 815, 67, 104, 58, 512, 24, 892, 894, 767, 553
9 , 81, 379, 843, 831, 445, 742, 717,
10 958, 609, 842, 451, 688, 753, 854, 685, 93,
11 857, 448, 380, 126, 721, 328, 753, 470,
12 743, 527
13
14 # your code goes here
15 for number in numbers:
16     if number == 237:
17         break
18     if number % 2 == 0:
```

Great work!

Solution Submit

Exercise

In this exercise you'll use an existing function, and while adding your own to create a fully functional program.

1. Add a function named `list_benefits()` that returns the following list of strings: "More organized code", "More readable code", "Easier code reuse", "Allowing programmers to share and connect code together".
2. Add a function named `build_sentence(benefits)`, which receives a single argument containing a string and returns a sentence starting with the given string and ending with the string " is a benefit of functions!"
3. Run and see all the functions work together!

```
script.py
1 # Modify this function to return a list of
2 strings as defined above
3 def list_benefits():
4     return "More organized code", "More readable
5     code", "Easier code reuse", "Allowing programmers
6     to share and connect code together"
7
8 # Modify this function to concatenate to each
9 benefit - " is a benefit of functions!"
10 def build_sentence(benefit):
11     return "%s is a benefit of functions!" % benefit
12
13 def name_the_benefits_of_functions():
14     list_of_benefits = list_benefits()
15     for benefit in list_of_benefits:
```

Nice work!

Solution Submit

Exercise

We have a class defined for vehicles. Create two new vehicles called car1 and car2. Set car1 to be a red convertible worth \$60,000.00 with a name of Fer, and car2 to be a blue van named Jump worth \$10,000.00.

```
script.py
9 (self.name, self.color, self.kind, self.value)
10 return desc_str
11
12 # your code goes here
13 car1 = Vehicle()
14 name = "Fer"
15 kind = "car"
16 color = "red"
17 value = 60000.00
18
19 car2 = Vehicle()
20 name = "Jump"
21 kind = "car"
22 color = "blue"
23 value = 10000.00
24
25 # test code
```

Great job!

Solution Submit

Exercise

Add "Jake" to the phonebook with the phone number 938273443, and remove Jill from the phonebook.

```
script.py
1 phonebook = {
2     "John" : 938477556,
3     "Jack" : 938377264,
4     "Jake" : 938273443
5 }
6 # your code goes here
7
8 # testing code
9 if "Jake" in phonebook:
10     print("Jake is listed in the phonebook.")
11
12 if "Jill" not in phonebook:
13     print("Jill is not listed in the phonebook.")
```

Nice work!

Solution Submit

Exercise

In this exercise, print an alphabetically sorted list of all the functions in the re module containing the word find.

```
script.py
1 import re
2
3 # your code goes here
4 find_numbers = []
5 for member in dir(re):
6     if "find" in member:
7         find_numbers.append(member)
8 print(sorted(find_numbers))
```

Great work!

Solution Submit

Exercise

Write a generator function which returns the Fibonacci series. They are calculated using the following formula: The first two numbers of the series is always equal to 1, and each consecutive number returned is the sum of the last two numbers. Hint: Can you use only two variables in the generator function? Remember that assignments can be done simultaneously. The code

```
script.py
1 a = 1
2 b = 2
3 a, b = b, a
4 print(a, b)
```

IPython Shell

In [1]:

Run

will simultaneously switch the values of a and b.

```
script.py  solution.py
1 def fib():
2     a, b = 1, 1
3     while 1:
4         yield a
5         a, b = b, a + b
6 import types
7 if type(fib()) == types.GeneratorType:
8     print("Good, The fib function is a generator.")
9
10 counter = 0
11 for n in fib():
12     print(n)
13     counter += 1
14     if counter == 10:
15         break
```

IPython Shell

<script.py> output:

Good, The fib function is a generator.

1

1

2

3

5

8

13

21

34

55

Good work!

Submit

Exercise

Fill in the `foo` and `bar` functions so they can receive a variable amount of arguments (1 or more). The `foo` function must return the amount of extra arguments received. The `bar` must return `True` if the argument with the keyword `magicnumber` is worth 7, and `False` otherwise. [Computer Science](#)

```
script.py  solution.py
1 def foo(a, b, c, *args):
2     return len(args)
3
4 def bar(a, b, c, **kwargs):
5     return kwargs.get("magicnumber") == 7
6
7 if foo(1, 2, 3, 4) == 1:
8     print("Good.")
9 if foo(1, 2, 3, 4, 5) == 2:
10    print("Better.")
11 if bar(1, 2, 3, magicnumber=6) == False:
12    print("Great.")
13 if bar(1, 2, 3, magicnumber=7) == True:
14    print("Awesome!")
```

IPython Shell

<script.py> output:

Good.

Better.

Great.

Awsome!

In [1]:

Great work!

Submit

In the exercise below, use the given lists to print out a set containing all the participants from event A which did not attend event

[Learn Coding Online](#)

```
script.py
1 a = set(["Jake", "John", "Eric"])
2 b = set(["John", "Jill"])
3 print(a.difference(b))
4 print(b.difference(a))
```

IPython Shell

<script.py> output:

['Jake', 'Eric']

['Jill']

In [1]:

Nice work!

Solution

Submit

The aim of this exercise is to print out the JSON string with key-value pair "Me": 800 added to it. [Java \(Programming Language\)](#)

```
script.py
1 import json
2 def add_employee(salaries_json, name, salary):
3     salaries = json.loads(salaries_json)
4     salaries[name] = salary
5
6     return json.dumps(salaries)
7
8 salaries = '{"Alfred": 300, "Jane": 400}'
9 new_salaries = add_employee(salaries, "Me", 800)
10 decoded_salaries = json.loads(new_salaries)
11 print(decoded_salaries["Alfred"])
12 print(decoded_salaries["Jane"])
13 print(decoded_salaries["Me"])
```

IPython Shell

<script.py> output:

300

400

800

In [1]:

Great work!

Solution

Submit

Exercise

Using a list comprehension, create a new list called "newlist" out of the list "numbers", which contains only the positive numbers from the list, as integers. [Python Programming](#)

```
script.py
1 numbers = [34.6, -203.4, 44.9, 68.3, -12.2, 44.6, 12.7]
2 newlist = [int(num) for num in numbers if num > 0]
3 print(newlist)
```

IPython Shell

<script.py> output:

[34, 44, 68, 44, 12]

In [1]:

Very nice!

Solution

Submit

Restart Session

Exercise

Write a program using lambda functions to check if a number in the given list is odd. Print "True" if the number is odd or "False" if not for each element.

```
script.py
1 l = [2,4,7,3,14,19]
2 for i in l:
3     # your code here
4     my_lambda = lambda x: (x % 2) == 1
5     print(my_lambda(i))
```

IPython Shell

<script.py> output:

False

False

True

True

False

True

In [1]:

Nice work!

Solution

Submit

Exercise

Handle all the exception! Think back to the previous lessons to return the last name of the actor.

```
script.py
1 # Setup
2 actor = {"name": "John Cleese", "rank": "awesome"}
3
4 # Function to modify!!!
5 def get_last_name():
6     return actor["name"].split()[1]
7
8 # Test code
9 get_last_name()
10 print("All exceptions caught! Good job!")
11 print("The actor's last name is %s" % get_last_name())
```

IPython Shell

<script.py> output:

All exceptions caught! Good job!

The actor's last name is Cleese

In [1]:

Great work!

Solution

Submit

Exercise

Edit the function provided by calling `partial()` and replacing the first three variables in `func()`. Then print with the new partial function using only one input variable so that the output equals 60.

```
script.py
1 #Following is the exercise, function provided:
2 from functools import partial
3 def func(u, v, w, x):
4     return u*4 + v*3 + w*2 + x
5
6 #Enter your code here to create and print with your partial function
7 p = partial(func,5,6,7)
8 print(p(8))
```

IPython Shell

<script.py> output:

60

In [1]:

Good job!

Solution

Submit

CONCLUZII

Parcurgerea completă a acestui modul introductiv asigură asimilarea deprinderilor fundamentale necesare dezvoltării de software și scrierii de scripturi în limbajul Python. Prin utilizarea mediului interactiv Codédex, au fost aprofundate aspecte esențiale precum sintaxa specifică, declararea variabilelor, gestionarea tipurilor de date și implementarea logicii de control, toate validate prin execuție directă în browser.

Competențele dobândite în cadrul acestei lucrări constituie o bază critică pentru tranziția de la limbaje cu un nivel de abstractizare mai scăzut către limbaje de nivel înalt. Fluența în scrierea și depanarea scripturilor Python reprezintă un pilon esențial în pregătirea tehnică modernă, facilitând abordarea ulterioară a modulelor avansate de automatizare a sistemelor, securitate cibernetică sau analiză a datelor. Astfel, prin finalizarea exercițiilor practice, ați pus fundația necesară pentru a utiliza Python ca pe un instrument versatil și puternic în soluționarea problemelor tehnice.

BIBLIOGRAFIE

1.